

PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

[Página inicial](#) ▶ [Meus cursos](#) ▶ [PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2](#) ▶ [Tópico 14](#) ▶

[Exercício: algoritmos gulosos](#)

Exercício: algoritmos gulosos?

Fase de avaliação

Fase de configuração	Fase de envio	Fase de avaliação	Fase de cálculo da nota da avaliação	Encerrado
	Envie seu trabalho Prazo limite dos envios: quinta, 1 dez 2022, 23:59 (1324 dias atrás)	Fase atual ● Prazo limite da avaliação: quinta, 8 dez 2022, 23:59 (1317 dias atrás)		

Seu envio ▼

Você não enviou seu trabalho ainda

Instruções para avaliação ▼

Respostas:

1. a. Seja O' a solução obtida pela troca de v_k por u_k em O . A solução O' também é ótima, pois possui o mesmo número de hotéis da solução O . Resta mostrar que O' é viável.

Primeiro observe que $v_{k-1} < u_k \leq v_{k+1}$. Como $v_{k-1} = u_{k-1}$ e $u_{k-1} < u_k$, concluímos que $v_{k-1} < u_k$. Temos também que $u_k \leq v_{k+1}$, pois caso contrário conseguiríamos uma solução melhor que a solução ótima O removendo o hotel v_{k+1} da solução O' .

Como o hotel u_k é encaixado entre os hotéis v_{k-1} e v_{k+1} , para que O' seja viável basta que

- i. $x[u_k] - x[v_{k-1}] \leq D$, e
- ii. $x[v_{k+1}] - x[u_k] \leq D$.

A condição (i) decorre do fato de $v_{k-1} = u_{k-1}$ e da solução gulosa G ser viável ($x[u_k] - x[u_{k-1}] \leq D$).

Como $v_{k-1} = u_{k-1}$, $x[v_k] - x[v_{k-1}] \leq D$ (pois O é viável) e u_k é o hotel mais distante de u_{k-1} que não ultrapassa D km, concluímos que $x[v_k] \leq x[u_k]$. Multiplicando esta desigualdade por -1 e somando com $x[v_{k+1}]$ obtemos $x[v_{k+1}] - x[u_k] \leq x[v_{k+1}] - x[v_k]$. Como $x[v_{k+1}] - x[v_k] \leq D$ (pois O é viável), temos a condição (ii).

b.

```
algoritmo parada_em_hotéis(x[1..n]):
Entrada: Lista x[1..n] contendo a posição (em km) dos hotéis 1..n ao longo da estrada.
         Os valores em x estão em ordem crescente.
Saída:   Lista G contendo as paradas em hotéis de uma solução ótima (gulosa).

1  G = [1] // primeiro hotel           0(1)
2  x_ini = x[1] // x_ini é a posição inicial 0(1)
3  for i de 2 até n-1                   0(n)
4      if x[i+1] - x_ini > D             0(n)
5          G.append(i)                   0(n)
6          x_ini = x[i]                   0(n)
7  G.append(n) // último hotel          0(1)
8  return G                             0(1)
```

Como o for da linha 3 tem $n - 2$ iterações, e cada execução das instruções tem tempo constante, concluímos que o algoritmo é $T(n) = O(n)$.

Envios atribuídos para avaliação ▼

Você não recebeu nenhum envio para avaliar

×

◀ Vídeo: algoritmos gulosos (40min)

Seguir para...

Slides: algoritmos gulosos ▶